

DVWA Writeup

I started by clicking around and exploring the website getting used to the structure of DVWA. My first instinct was to check for SQL injections since they are pretty common, and found a vulnerable query.

Vulnerability: SQL Injection

User ID:

Custom SQL Injection File Loaded

User	PW	F	L
a	5f4d	a	a

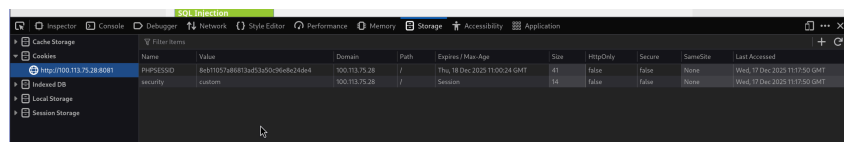
Vulnerability: SQL Injection

User ID:

Custom SQL Injection File Loaded

No results found.

I tried some more manual sql injection attempts, but didn't manage to get further from here. I tried to use sql-map to see if that could find a vulnerability, but didn't manage to get very far. Using the website inspector I noticed there was a session cookie hoping that was the issue , but still couldn't get further.



Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
PHPSESSID	8eb11057a86813ad53a50c96e8e24de4	100.113.75.28	/	Thu 18 Dec 2025 11:00:24 GMT	81	True	False	None	Wed 17 Dec 2025 11:17:50 GMT
security	custom	100.113.75.28	/		14	False	False	None	Wed 17 Dec 2025 11:17:50 GMT

```
[12:26:08] [WARNING] GET parameter 'id' does not seem to be injectable
[12:26:08] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'

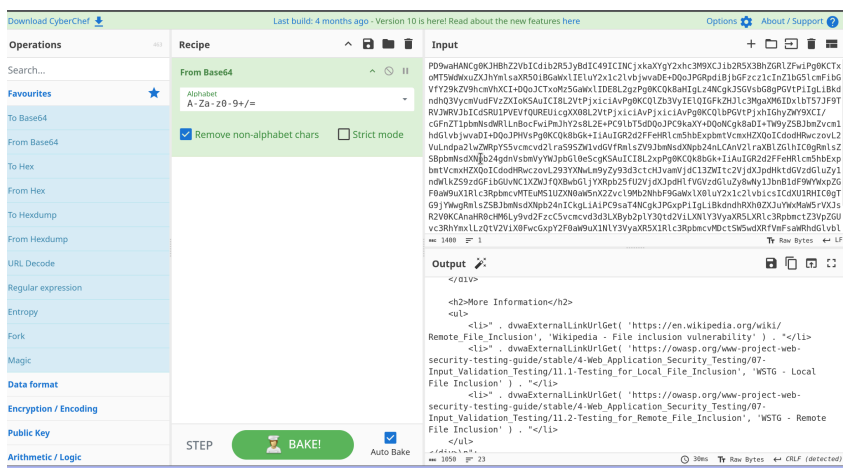
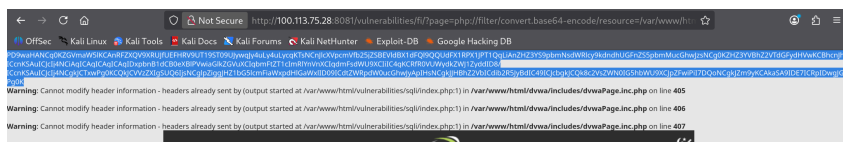
[*] ending @ 12:26:08 /2025-12-17/

mago@asahi ~-> sqlmap --flush-session -u "http://100.113.75.28:8081/vulnerabilities/sqli/?role=&debug=&id=&Submit=Submit#" --cookie "PHPSESSID=8eb11057a86813ad53a50c96e8e24de4" -p id --level 3
```

I then tried Local File Inclusion thinking that in the case where this is vulnerable, i could potentially get some more information about the script. I tried a simple php filter, and it worked!



With this, could now browse the files. From an error message disclosure earlier i could easily guess the folder structure of each part. From there i tried looking for exploits by feeding in every `<vuln/>source/custom.php` script into the base64 filter and decoding it with cybberchef.



I noticed that only some categories were vulnerable as lfi only worked for existing files. Some categories only had the impossible.php, while others had a custom. From here i started reading the source code for the lfi and sqli. I noticed that most of the vulnerable pages had a debug and role field i could add to the query. This made it easier to see what was going on, and also let me bypass some of the checks in some of the categories.

```

mysql_connect($mysqlHost, $mysqlUser, $mysqlPass);

$method = 'GET';
$vulnerabilityFile = '';
switch( dvwaSecurityLevelGet() ) {
    case 'low':
        $vulnerabilityFile = 'low.php';
        break;
    case 'medium':
        $vulnerabilityFile = 'medium.php';
        $method = 'POST';
        break;
    case 'high':
        $vulnerabilityFile = 'high.php';
        break;
    case 'custom':
        $vulnerabilityFile = 'custom.php';
        break;
    default:
        $vulnerabilityFile = 'impossible.php';
        break;
}

```

I tried portscanning the ip of the website to see if there were more services running. The credentials for the mysql database were hardcoded in the `custom.php` file. This meant if the server let me access the sql database directly i could simply log-in and potentially use execution queries to get a reverse shell. Unfortunately, no interesting ports other than the website were running so i stopped there.

```

mago@asahi ~> nmap 100.113.75.28
Starting Nmap 7.92 ( https://nmap.org ) at 2025-12-17 13:04 CET
Nmap scan report for macbook-pro-julia-2.tail540256.ts.net (100.113.75.28)
Host is up (0.0057s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
5000/tcp  open  upnp
7000/tcp  open  afs3-fileserver
8081/tcp  open  blackice-icecap
8082/tcp  open  blackice-alerts

```

I then tried messing around with XSS for a while, but nothing gave me results. I managed to turn off filtering completely using `role=admin`, but i couldn't get any php code to be run so i gave up.

```
// -----  
// STEP 4 – Hidden backdoor  
// role=admin disables filter  
// -----  
if ($role_flag === "admin") {  
    $skip_filter = true;  
}
```

From here i tried the file upload. Reading the PHP code, this seemed very useful. In theory as long as i could bypass all the checks, i could masquerade an html file with php as a png or a gif. since the gif had the least restrictions i tried this. I simply got a basic php shell script calling system() with a command of choice, then displayed the output on the page.

This worked and i had a reverse shell. I managed to find the flag file path pretty quick, but had no way of opening it as cat wasnt runnable. I tried different ways of opening files as well like piping into echo like `echo $(<flag.txt)$`, but that didnt work either. In the end i found `dd if=flag.txt` to work, and managed to print the flag contents.



GIF

FLAG{INITIAL_FOOTHOLD_ACHIEVED}